Milestone 2: Performance–Reliability Trade-offs in Heuristic, Supervised, and Imitation-Learned Path Planning

Suryaprakash Rajkumar

February 28, 2026

GitHub - https://github.com/suryaprakashrajkumar/AI-6450

1 Milestone-2 Scope and Deliverables

Based on the course requirements, this milestone addresses: (i) model selection and architectural justification, (ii) training and experimental reporting, (iii) efficiency metrics, and (iv) deployment-oriented design trade-offs. All codes is implemented in Python/PyTorch can be accessed on GitHub.

2 Methodology: End-to-End System

2.1 Problem Formulation

At time step t, the agent state s_t is represented by a local occupancy patch P_t, relative-goal vector g_t, and position p_t. The learning objective is multi-task: predict the next action $a_t \in \{0, \ldots, 7\}$ (classification) and the remaining cost-to-go c_t (regression).

2.2 Dataset Generation Strategy

We procedurally generate maze maps with controlled complexity and a random seed for reproducibility. Valid start-goal pairs are sampled only when reachable. An 8-connected A* planner produces expert demonstrations and optimal costs.

Each trajectory is converted to step-level supervised records:

- Local occupancy patch centered on the agent,
- Relative-goal displacement,
- Agent coordinates,
- Expert next action label (8-class),
- Remaining optimal cost-to-go label.

2.3 Quality and Issue Handling Plan

A dataset audit module validates the assumptions from Milestone 1: missing values, unit consistency, duplicate states, class imbalance, outliers, and train/test drift. In the current audited set, missing values were absent, occupancy values were binary, and drift was low, indicating acceptable supervision quality for baseline model training.

2.4 Training and Inference Pipeline

A shared MLP backbone with two task heads is used:

$$h_t = f_\theta(x_t), \qquad z_t = W_a h_t + b_a, \qquad \hat{c}_t = W_c h_t + b_c.$$

The action head outputs logits $z_t \in \mathbb{R}^8$, and the cost head predicts scalar $\hat{c}_t$. During rollout, invalid actions are masked before selection. This jointly supports control decisions (classification head) and value estimation (regression head).

The end-to-end procedure is executed in the following sequence:

1. **Configuration and seeding:** set grid size, maze parameters, split ratios, and random seeds for reproducibility.
2. **Maze generation and sampling:** produce procedural maps and sample valid start-goal pairs.
3. **Expert labeling with A*:** compute optimal trajectories and step-wise cost-to-go labels.
4. **Feature/label extraction:** convert each trajectory step into supervised records (patch, relative goal, position, action, cost).
5. **Quality assurance:** run audits for missing values, duplicates, class balance, outliers, and train-test drift.
6. **Model training:** train a PyTorch multi-task MLP for 100 epochs and save checkpoints, logs, and curves.
7. **Closed-loop inference:** deploy policy on unseen mazes with invalid-action masking and trajectory logging.
8. **System benchmarking:** compare against A* using success rate, optimality ratio, and per-decision latency, plus auxiliary metrics.

3 Model Selection and Justification

3.1 Plausible Model Families

Three plausible families were considered:

1. **Linear / Logistic models**: fast and interpretable, but limited for maze topology.
2. **Tree-based methods**: robust on tabular features, weaker on local spatial occupancy patterns without heavy feature engineering.

3. **Neural MLP / CNN**: stronger non-linear capacity and straightforward multi-task extension; higher training cost but still real-time feasible at inference.

3.2 Chosen Baseline: Multi-task MLP

The MLP is selected because it offers the best baseline balance of expressiveness, simplicity, and deployment speed. It directly matches the mixed objective (classification + regression), reuses a shared latent representation, and keeps inference lightweight enough for per-step robotic decision support.

4 Training and Experimentation

4.1 Split Strategy and Training Setup

Data is split into train/validation/test sets at dataset generation time with fixed seeds for reproducibility. Training is performed for 100 epochs using PyTorch, with logged learning curves and saved checkpoints. A class-weighted hybrid action loss was also explored to mitigate mild action imbalance.

4.2 Learning Dynamics

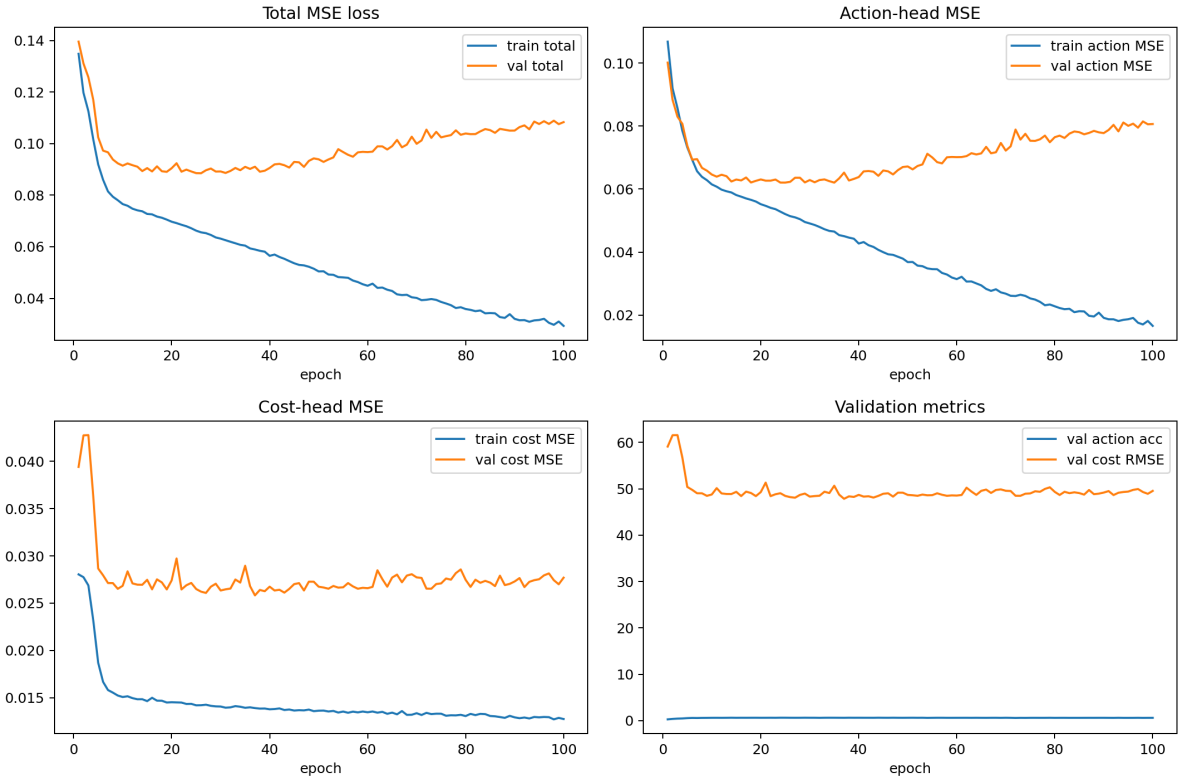
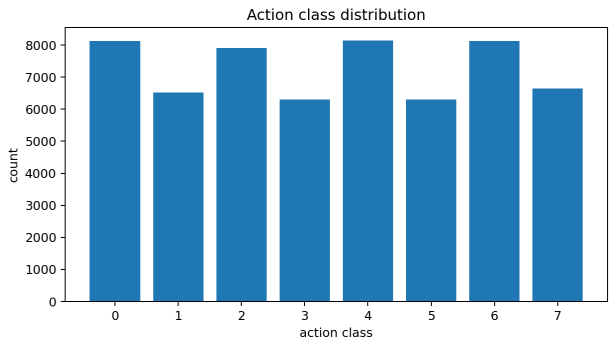
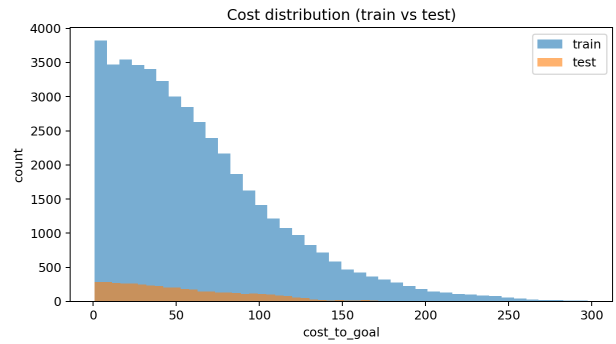


Figure 1: Training dynamics of the multi-task model (total loss, task losses, and validation metrics). The curves show stable convergence but indicate a gap between per-step prediction quality and long-horizon closed-loop navigation success.

4.3 Dataset Diagnostics



(a) Action class distribution.



(b) Cost-to-go distribution (train vs test).

Figure 2: Dataset distribution visualizations used to verify class balance and distribution shift assumptions before training.

5 Inference Behavior and System-Level Evaluation

5.1 Joint Inference (Classification + Regression)

At each step, one forward pass outputs both action logits and cost estimate:

$$\pi(a \mid s_t) = \mathrm{softmax}(z_t), \qquad a_t = \arg\max_{a \in \mathcal{A}_{\mathrm{valid}}(s_t)} \pi(a \mid s_t), \qquad \hat{c}_t \in \mathbb{R}.$$

The action head drives movement, while regression provides value context for monitoring or future value-guided action refinement.

5.2 Qualitative Trajectory Example

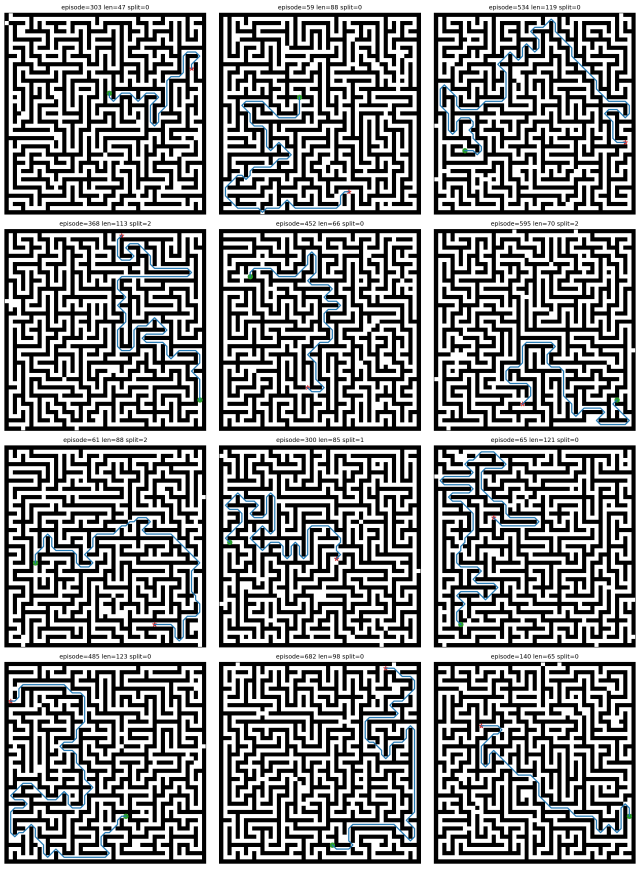


Figure 3: Example maze trajectory visualization, the image showcases some example of the trajectories collected using A*

6 Current Baseline Results Discussion

6.1 Quantitative Summary

Table 1 reports the main baseline metrics from the current multi-task MLP run on the small-maze setting.

Table 1: Current baseline performance summary (small-maze setting).

Metric	**Torch MLP**	**A***
Success rate	0.177	1.000
Optimality ratio	5.007	1.000
Decision time per step (ms)	0.215	0.020
Test action accuracy	0.645	–
Test cost RMSE	33.53	–
Collision rate	0.000	–
Timeout rate	0.823	0.000

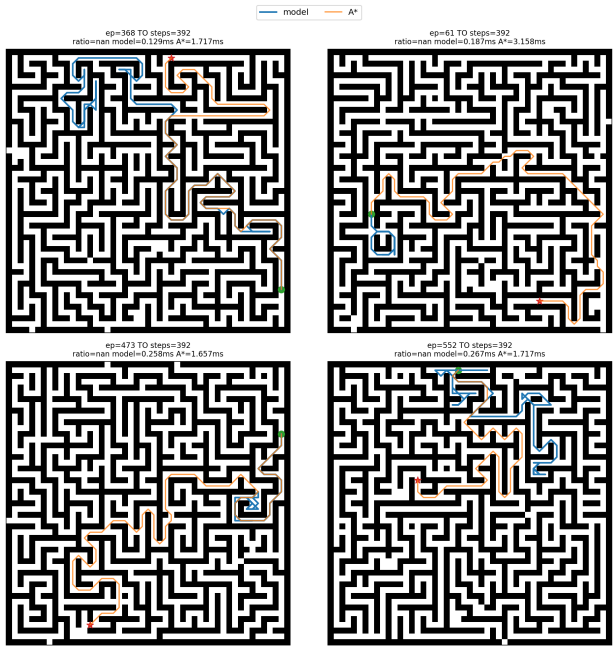


Figure 4: Inference with 4 cases

*The model is not trained to run on CUDA, can expect better in the final verison in terms of performance. The performance is iteratively should immprove

6.2 Interpretation

The baseline confirms a common behavior-cloning limitation in long-horizon navigation. While per-step predictive quality is moderate (action accuracy $\approx 64.5\%$), closed-loop reliability remains low due to compounding errors: the policy frequently enters suboptimal loops and times out before reaching the goal. This explains the gap between acceptable step-wise metrics and weak episode-level KPIs (success and optimality).

From a systems perspective, the learned policy remains computationally lightweight, but A* still dominates reliability and path quality. Therefore, the current baseline is best interpreted as a deployment-oriented starting point that validates the pipeline and motivates the imitation-learning upgrade proposed in the next section.

7 Efficiency and Deployment Considerations

7.1 Training and Inference Efficiency

Training and inference logs are stored as artifacts for reproducibility. Inference is lightweight (sub-millisecond decision latency on CPU in this environment), but closed-loop quality remains the primary bottleneck compared with A* optimality.

7.2 Deployment Design and Trade-offs

The intended deployment is an **online edge inference + planner fallback** pattern:

- Learned policy handles frequent low-latency action proposals.
- A* fallback is triggered under uncertainty, repeated revisits, or timeout risk.
- This hybrid design balances responsiveness and safety under runtime constraints.

8 Conclusion and Next Steps

Milestone 2 demonstrates a complete, reproducible learning pipeline from synthetic expert data to system-level KPI evaluation. The selected MLP baseline is suitable for fast iteration and deployment-oriented analysis, but current closed-loop reliability in complex mazes indicates the need for stronger sequential robustness (e.g., data aggregation, value-guided action selection, or richer policy architectures) in the next milestone.

9 Next Step: Imitation Learning Paradigm and Improvement Plan

The current supervised policy is affected by compounding error: once rollout states deviate from expert trajectories, the timeout probability increases. Imitation learning is the next logical step because it reduces this train-test state mismatch.

9.1 Proposed Paradigm

We will keep the same state representation (P_t, g_t, p_t), action space (8-neighbor), and multi-task network, but move from one-shot behavior cloning to dataset aggregation:

- Roll out the current learner on mazes,
- Query A* for corrective labels on visited learner states,
- Aggregate new samples and retrain iteratively.

9.2 Improvement Plan

1. **Data:** increase hard-case coverage (dead-ends, narrow corridors, long-horizon routes) and version datasets by iteration.
2. **Training:** keep hybrid action/regression objective, class weighting, and validation-based tuning.
3. **Inference:** retain valid-action masking and add confidence-based A* fallback for safety.

9.3 Execution and Expected Impact

Planned execution: (i) reproduce behavior-cloning baseline, (ii) run iterative aggregation rounds, and (iii) benchmark against A* and Milestone-2 baseline. Expected impact: higher success rate, lower timeout rate, and improved optimality ratio while preserving low-latency inference.